

In [2]:

```

# =====
# EXPERIMENT 4A – SVM (RBF)
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(
    data.target,
    name="target"
)

# 0 = Malignant
# 1 = Benign

print("Dataset Shape:", X.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

svm = SVC(

```

```

        kernel='rbf',
        probability=True,
        random_state=42
    )

    param_grid = {
        'C': [0.1, 1, 10, 100],
        'gamma': ['scale', 0.01, 0.001, 0.0001]
    }

    grid = GridSearchCV(
        estimator=svm,
        param_grid=param_grid,
        scoring='f1',
        cv=5,
        n_jobs=-1
    )

    grid.fit(X_train_sc, y_train)

    print("\nBest Parameters:")
    print(grid.best_params_)

    best_svm = grid.best_estimator_
    y_pred = best_svm.predict(X_test_sc)

    y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)

    cm = confusion_matrix(y_test, y_pred)

    print("\n==== SVM Test Metrics =====")

    print("Accuracy :", round(acc, 4))
    print("Precision:", round(prec, 4))
    print("Recall   :", round(rec, 4))
    print("F1 Score  :", round(f1, 4))
    print("ROC AUC   :", round(auc, 4))

    print("\nConfusion Matrix:")
    print(cm)

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # 9. Confusion Matrix Heatmap
    plt.figure(figsize=(6,5))

    import seaborn as sns

    sns.heatmap(

```

```
cm,
annot=True,
fmt='d',
cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)

plt.figure(figsize=(7,5))

plt.plot(
    fpr,
    tpr,
    label=f"SVM (AUC = {auc:.3f})"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - SVM")
plt.legend()
plt.grid(True)

plt.show()
```

Dataset Shape: (569, 30)

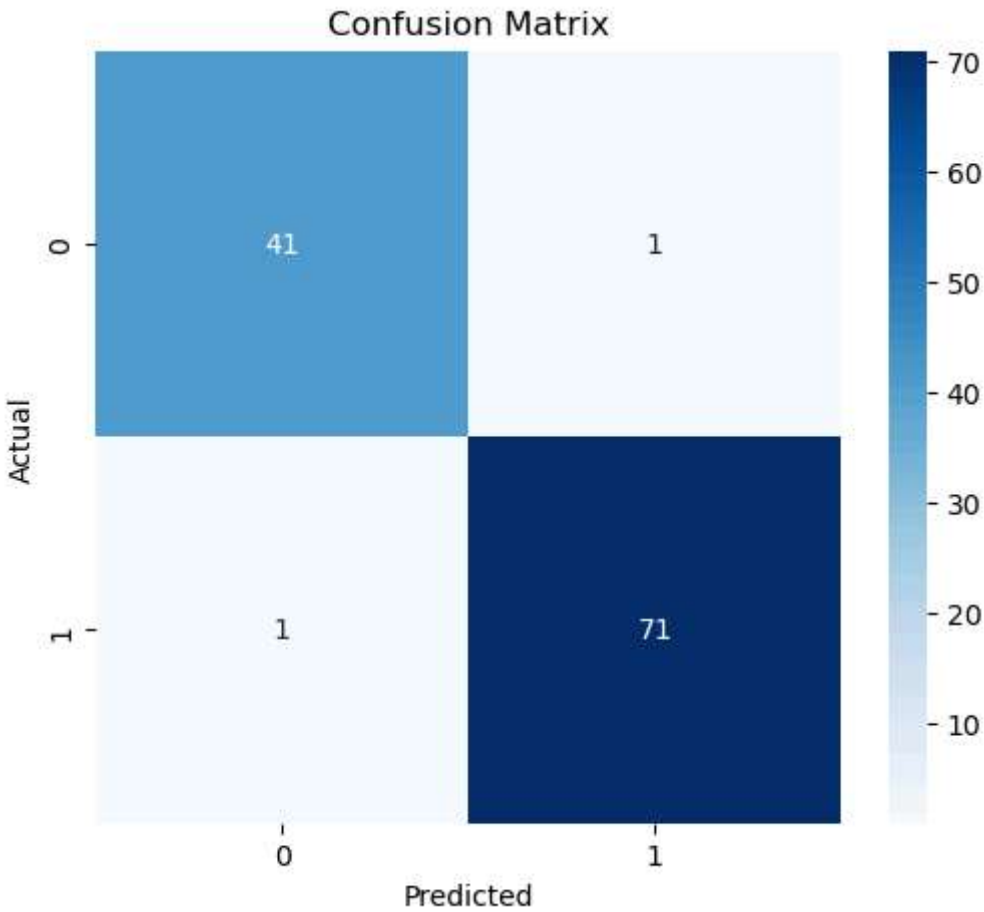
Best Parameters:
{'C': 10, 'gamma': 0.01}

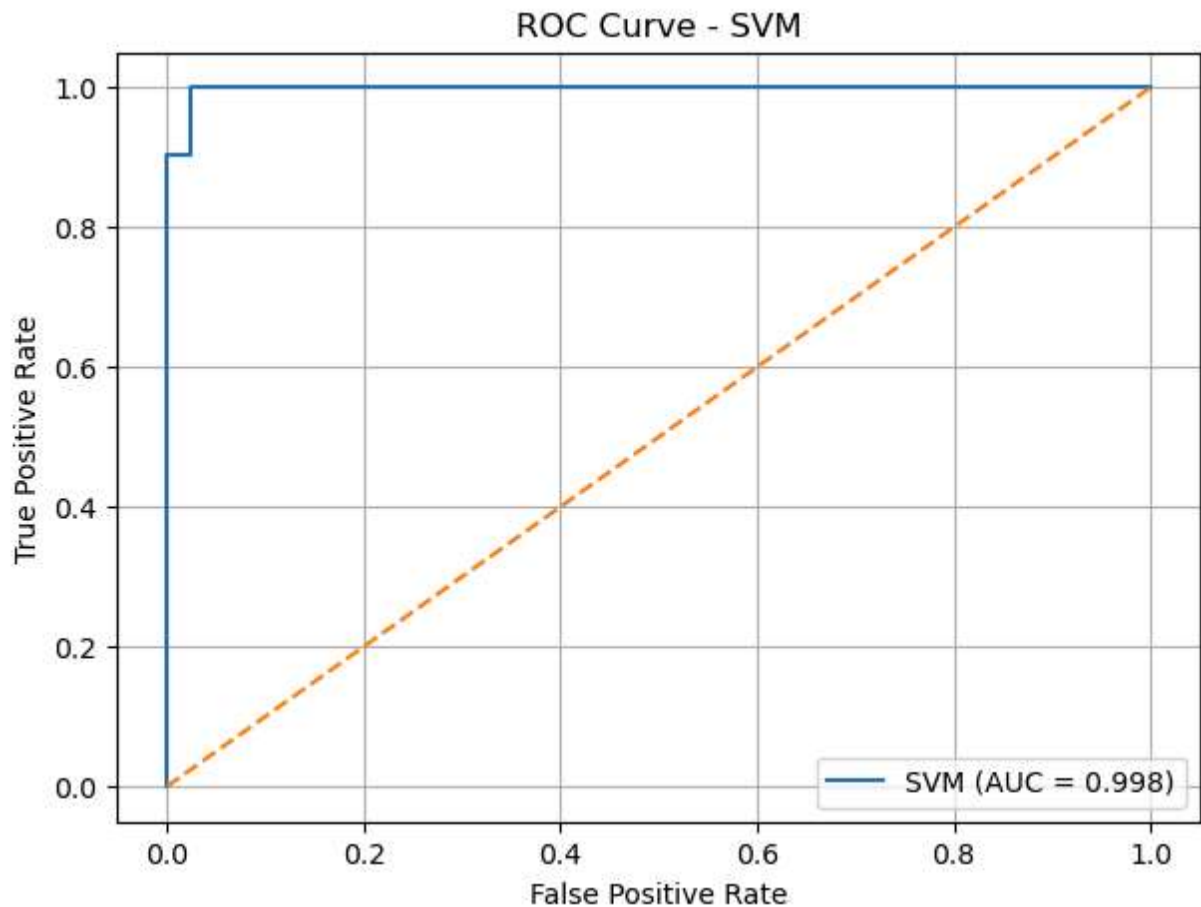
==== SVM Test Metrics ====
Accuracy : 0.9825
Precision: 0.9861
Recall : 0.9861
F1 Score : 0.9861
ROC AUC : 0.9977

Confusion Matrix:
[[41 1]
 [1 71]]

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114





In [3]:

```
# =====
# PROJECT Customer Churn Prediction Using
# =====

#Customer Churn Prediction Using Support Vector Machine

#AIM

#To predict whether a customer will leave a service or continue using it based on c

#OBJECTIVE
### Build an SVM model for customer churn prediction.
#Classify customers as Churn or Not Churn.
#Evaluate model performance using Accuracy and Confusion Matrix.
#Visualize classification results.
```

In [4]: **import** pandas **as** pd

```
data = {
    'MonthlyCharges': [20,85,50,95,40,78,35,90,45,88,30,92,55,80,38,87,42,91,48,84],
    'Tenure': [24,5,18,3,30,7,28,4,20,6,36,2,15,8,25,5,22,3,19,7],
    'SupportCalls': [1,5,2,6,1,4,1,5,2,5,0,6,3,4,1,5,2,6,2,4],
    'Churn': [0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1]
}

df = pd.DataFrame(data)
```

```
print(df.head())
```

	MonthlyCharges	Tenure	SupportCalls	Churn
0	20	24	1	0
1	85	5	5	1
2	50	18	2	0
3	95	3	6	1
4	40	30	1	0

```
In [5]: X = df[['MonthlyCharges', 'Tenure', 'SupportCalls']]
        y = df['Churn']
```

```
In [6]: # =====
# EXPERIMENT 4A – SVM (RBF)
# =====

# 1. Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

# 2. Load Dataset
data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(
    data.target,
    name="target"
)

# 0 = Malignant
# 1 = Benign

print("Dataset Shape:", X.shape)

# 3. Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4. Feature Scaling
scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# 5. SVM Model
svm = SVC(
    kernel='rbf',
    probability=True,
    random_state=42
)

# 6. Hyperparameter Tuning
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.01, 0.001, 0.0001]
}

grid = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1
)

grid.fit(X_train_sc, y_train)

print("\nBest Parameters:")
print(grid.best_params_)

best_svm = grid.best_estimator_

# 7. Prediction
y_pred = best_svm.predict(X_test_sc)

y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

# 8. Evaluation Metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)

cm = confusion_matrix(y_test, y_pred)
```

```

print("\n==== SVM Test Metrics =====")

print("Accuracy :", round(acc, 4))
print("Precision:", round(prec, 4))
print("Recall   :", round(rec, 4))
print("F1 Score :", round(f1, 4))
print("ROC AUC  :", round(auc, 4))

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 9. Confusion Matrix Heatmap
plt.figure(figsize=(6,5))

import seaborn as sns

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# 10. ROC Curve
fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)

plt.figure(figsize=(7,5))

plt.plot(
    fpr,
    tpr,
    label=f"SVM (AUC = {auc:.3f})"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - SVM")
plt.legend()
plt.grid(True)

```



```
plt.show()
```

Dataset Shape: (569, 30)

Best Parameters:

{'C': 10, 'gamma': 0.01}

===== SVM Test Metrics =====

Accuracy : 0.9825

Precision: 0.9861

Recall : 0.9861

F1 Score : 0.9861

ROC AUC : 0.9977

Confusion Matrix:

```
[[41  1]
```

```
 [ 1 71]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

